Erick Epley

August 11th, 2026

IT FDN 110 GP Su 26

Assignment 07

Github URL (external site)

Class Inherience and Objects

Introduction

This week I learned how to create new classes that inherit traits and functions from other classes. With this assignment I worked to create two new classes: Person and Student, which I would use to create new properties and objects for their respective classes. I then ensured that previous functions and classes were updated to utilize my new knowledge.

Creating New Classes

To start this week's program, I began with the starter file for the assignment which I renamed to the correct filename in the acceptance criteria: “Assignment07”. This starter file already had the previous FileProccesor class and IO class created in previous assignments which were in place to process file data and process a series of input and output commands from the user. For this assignment, I would need to create 2 more classes: the Person class and the Student class.

Since the Student class was meant to inherit from the Person class, I started the code by creating the Person class. I added a descriptive comment string, then I used the “def” keyword and the __init__ constructor to initialize self, as well as the first_name and last_name variables as strings. (Figure 1).

```python
# Begin Person Class
class Person:  1 usage
    """
    A class to hold Person data

    ChangeLog: (Who, When, What)
        Erick Epley, 8/10/2026, Created Class
    """

    # Add first_name and last_name properties to the constructor
    def __init__(self, first_name: str = "", last_name: str = ""):
        self.first_name = first_name
        self.last_name = last_name
```

Figure 1. Beginning the Person Class

Now that I had initialized the first_name and last_name variables I could create properties for each of them. To do this I would need to create a "getter" and a "setter" which would create a way for me to retrieve the information from the first_name or last_name variables based on the data passed through the class. I used the @property method to create the getter. I created a function using def first_name(self), then returned the first_name with the .capitalize() command which would ensure the name was capitalized. (Figure 2).

```python
# Getter and setter for the first_name property
@property  4 usages (2 dynamic)
def first_name(self):
    return self.__first_name.capitalize()
```

Figure 2. The first_name property method

Next I created the first_name setter, using the @first_name.setter method, which I could pass "self" through as well as a value of a string. I added an if statement to check and see if the first name had been written with alphabetical characters and not numbers or symbols using the .isalpha() command and raised an error in the case of the name not being alphabetical.

```python
# first_name setter
@first_name.setter  3 usages (2 dynamic)
def first_name(self, value: str):
    if value.isalpha() or value == "":  # if alphabet or empty string
        self.__first_name = value
    else:
        raise ValueError("The first name should not contain numbers. ")
```

Figure 3. The first_name.setter method

I repeated this process for the last_name variable as well, creating a getter and setter which would allow me to call on the class in the future, or as we will see later in this project allow the Student class to call all of these methods. (Figure 4).

```
# Getter and setter for the last_name property
@property  4 usages (2 dynamic)
def last_name(self):
    return self.__last_name.capitalize()

# last_name setter
@last_name.setter  3 usages (2 dynamic)
def last_name(self, value: str):
    if value.isalpha() or value == "":
        self.__last_name = value
    else:
        raise ValueError("The last name should not contain numbers. ")
```

Figure 4. The last_name getter and setter methods

The last thing I added to the Person class was a way to override the __str__() method to return a Person's data. I created a function with the __str__() method, passing self through it and returning an f string that called on the first_name and last_name through self, organized as a comma separated value string. (Figure 5).

```
# Override the __str__() method to return Person data
# as a comma separated string
def __str__(self):
    return f"{self.first_name},{self.last_name}"
```

Figure 5. Overriding the __str__() method

With the Person class done, I moved onto the Student class. This class would inherit the methods from the Person class, allowing me to call on them through the Student class itself while also adding new methods that could be used for student course information that the Person class wasn't concerned with. I initialized the variables with the __init__() method for the Student class after passing the Person class through the initial line of code, and then I used the super() function which would call on the Person class, which the Student class was now a subclass of. I also initialized the course_name variable by calling on self.course_name. (Figure 6).

(Root, Randal. *Introduction to Programming with Python*. Self, 2026, p. Module 07.)

```python
# Begin Student Class
class Student(Person):  3 usages
    """
    A class to hold Student data and inherits from Person class

    Properties:
        first_name: str
        last_name: str
        course_name: str

    ChangeLog: (Who, When, What)
        Erick Epley, 8/10/2026, Created Class
    """

    # call to the Person constructor and pass it the first_name and last_name data
    def __init__(self, first_name: str = "", last_name: str = "", course_name: str = ""):
        super().__init__(first_name=first_name, last_name=last_name)
```

Figure 6. The beginning of the Student class

I followed a similar pattern from earlier, using the same @property method and .setter method to create a getter and setter for the course_name string, only this time not including the if statement since a course name could have numbers and symbols as well as alphabetical characters. (Figure 7). During this section of the project I ran into a big error message, telling me that the course_name variable had an unresolved attribute. After a bit of debugging, I found out that I had accidentally written the “last_name” variable instead of the “course_name” variable in the setter for the course_name, and after correcting that issue the code was running properly again.

```python
# add the getter for course_name
@property  5 usages (2 dynamic)
def course_name(self) -> str:
    return self.__course_name

# add the setter for course_name
@course_name.setter  4 usages (3 dynamic)
def course_name(self, value: str):
    self.__course_name = value
```

Figure 7. The course_name getter and setter

Lastly I overrode the __str__() method again, this time for the Student class, and used an f string to format the relevant data into a comma separated value string. (Figure 8).

```
# Override the __str__() method to return the Student data
# as a comma separated string
def __str__(self):
    return f"{self.first_name},{self.last_name},{self.course_name}"
```

Figure 8. Overriding the __str__() method

With these two classes now done I could call on them when inputting student data to organize it into properly formatted strings, then once again find that data and use it to write to a file, print to the user, or for any other task I needed.

Modifying Functions

With two new classes created, I was going to need to modify some of the other functions in the FileProcessor and IO classes that were created in the previous assignment to properly use the new Person and Student classes. The first step was to go to the read_data_from_file function, which would be the first thing the program called. Within the try statement, I needed to convert dictionary data loaded from the json file to list data for the students list. I used a for loop to look for student in json_students which would find a student_object of the data type Student (the class created earlier) and set that equal to a list of data that iterated through the FirstName, LastName, and CourseName of each student in the file. I appended that student_data to the student_object to ensure the data from the file would be saved in the list passed through the function. Lastly I closed the file to make sure it was saved. (Figure 9).

```
try:
    # Get a list of dictionary rows from the data file
    file = open(file_name, "r")
    json_students = json.load(file)

    # Convert the list of dictionary rows into a list of Student objects
    student_objects = []
    # convert dictionary data to Student data
    for student in json_students:
        student_object: Student = Student(first_name = student["FirstName"],
                                          last_name = student["LastName"],
                                          course_name = student["CourseName"])
        student_data.append(student_object)

    file.close()
```

Figure 9. Updated the read_data_from_file function

Next I followed a similar process for updating the write_data_to_file function, only this time in reverse. I used a for loop to look for student in student data within the json_student dictionary. I then appended the dictionary of json_student to the list_of_dictionary_data which added that converted list to the JSON file as a dictionary. (Figure 10). I made sure to still output the data to the user in case they wanted to see it and ensured the error handling properly worked as well.

```
try:
    # code to convert Student objects into dictionaries
    list_of_dictionary_data: list = []
    for student in student_data:
        json_student: dict = {"FirstName": student.first_name,
                              "LastName": student.last_name,
                              "CourseName": student.course_name}
        list_of_dictionary_data.append(json_student)

    file = open(file_name, "w")
    json.dump(list_of_dictionary_data, file, indent=2)
    file.close()

    IO.output_student_and_course_names(student_data=student_data)
```

Figure 10. Updated write_data_to_file function

Within the IO class, the output_student_and_course_names function needed to be updated. I used a for loop to iterate through the student_data. I created a new message variable that would organize the data into a comma separated value string, then used the .format() function to call student class for first_name, last_name, and course_name to fill the string with the correct data. This would then be printed out to the user if this function was called upon. (Figure 11).

```
print("-" * 50)
print("This is the current data: ")
for student in student_data:

    # access Student object data instead of dictionary data
    message = "{},{},{}."
    print(message.format(student.first_name, student.last_name, student.course_name))

print("-" * 50)
```

Figure 11. Updated output_student_and_course_names function

The last function to update was the input_student_data function, which would now need to call on the student class to properly set the first_name, last_name, and course_name data. Within the try statement, I set student to be equal to the Student() class, calling on it. I then used the input() command and called on the student class for each of the respective variables, allowing the user to populate them with data, then appended that data to the student_data list which could be passed through the function. (Figure 12).

```
try:
    # gain user input for student data while calling the Student() class
    student = Student()
    student.first_name = input("What is the student's first name? ")
    student.last_name = input("What is the student's last name? ")
    student.course_name = input("What is the student's current course? ")
    student_data.append(student)

    # print out the data input by the user
    print()
    print(f"{student.first_name} {student.last_name} has registered for {student.course_name}. ")
```

Figure 12. Updated input_student_data function

Finally, with all of the classes and functions updated I modified the function calls in each of the if statements to make sure the correct variables and lists were being passed through them for the code to properly function. (Figure 13).

```
# Present and Process the data
while (True):

    # Present the menu of choices
    IO.output_menu(menu=MENU)

    menu_choice = IO.input_menu_choice()

    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        students = IO.input_student_data(student_data=students)
        continue

    # Present the current data
    elif menu_choice == "2":
        IO.output_student_and_course_names(student_data=students)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        FileProcessor.write_data_to_file(file_name=FILE_NAME, student_data=students)
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop
    else:
        print("Please only choose option 1, 2, or 3")
```

Figure 13. The final While loop

Running the Program

With the program fully coded I ran it in both Pycharm as well as the command shell. It was functional in both, allowing me to read data from a file, add a new student to the roster, and write the full batch of data to a JSON file. (Figures 14-17).

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 1
What is the student's first name? Ezra
What is the student's last name? Bridger
What is the student's current course? Fencing 200

Ezra Bridger has registered for Fencing 200.
```

Figure 14. Output 1

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 2
-------------------------------------------------
This is the current data:
Peter,Parker,Biology 100.
Tony,Stark,Engineering 400.
Erick,Epley,Python 100.
Luke,Skywalker,Fencing 200.
Ted,Lasso,Soccer 100.
James,Kirk,Piloting 300.
Ezra,Bridger,Fencing 200.
-------------------------------------------------
```

Figure 15. Output 2

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 3
-------------------------------------------------
This is the current data:
Peter,Parker,Biology 100.
Tony,Stark,Engineering 400.
Erick,Epley,Python 100.
Luke,Skywalker,Fencing 200.
Ted,Lasso,Soccer 100.
James,Kirk,Piloting 300.
Ezra,Bridger,Fencing 200.
-------------------------------------------------
```

Figure 16. Output 3

```
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 4
Program Ended
```

Figure 17. Output 4

I checked my Enrollments.JSON file to see that it was also properly filled out and formatted with the correct data, including the new data I had input. (Figure 18).

```
[
  {
    "FirstName": "Peter",
    "LastName": "Parker",
    "CourseName": "Biology 100"
  },
  {
    "FirstName": "Tony",
    "LastName": "Stark",
    "CourseName": "Engineering 400"
  },
  {
    "FirstName": "Erick",
    "LastName": "Epley",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Luke",
    "LastName": "Skywalker",
    "CourseName": "Fencing 200"
  },
  {
    "FirstName": "Ted",
    "LastName": "Lasso",
    "CourseName": "Soccer 100"
  },
  {
    "FirstName": "James",
    "LastName": "Kirk",
    "CourseName": "Piloting 300"
  },
  {
    "FirstName": "Ezra",
    "LastName": "Bridger",
    "CourseName": "Fencing 200"
  }
]
```

Figure 18. Enrollments.JSON

I uploaded my Python script, Enrollments.JSON, and this knowledge document to a new Github repository at this link: https://github.com/erickmaster28/IntroToProg-Python-Mod07 (external site).

Summary

In summary, with this project I took skills I had learned about creating classes and functions in Python and added upon them with new methods like @propery and .setter, as well as how to allow one class to inherit methods, functions, and objects from another. I modified a program that allowed me to add students' data to a roster of first names, last names, and course names.

Works Cited

Arya, Anubhaw. *Mod07 - UsingProperties.* 2024.

Arya, Anubhaw. *Mod07 - UsingConstructors.* 2024.

Arya, Anubhaw. *Mod07 - UsingInheritance.* 2024.

Python Software Foundation. “3.14.6 Documentation.” *Python.Org*, 2019, https://docs.python.org/3/.

Root, Randal. *Introduction to Programming with Python*. Self, 2026, p. Module 07.